Regex2AFD: Compilador de Expresiones Regulares a Código C Optimizado para Entornos Embebidos

Hillebrand Giuliano, Sienra Daiana y Gonzalez Martin

Universidad Gastón Dachary, Posadas, Misiones, Argentina

Resumen Este trabajo presenta **Regex2AFD**, un compilador hecho en Python que toma una expresión regular y genera un autómata finito determinista (AFD) mínimo, junto con código C listo para usar en sistemas embebidos e IoT. El código generado no usa memoria dinámica ni librerías externas, y recorre la cadena en tiempo lineal $O(n)$. El frontend realiza el análisis léxico, sintáctico y semántico de la expresión regular, con una tabla de símbolos para las definiciones con nombre, y arma el Árbol de Sintaxis Abstracta (AST). Después, el motor aplica en orden Thompson, subconjuntos (agrupando los caracteres en clases de equivalencia) y minimización de Hopcroft. Las pruebas son reproducibles: Valgrind confirma que el código generado no hace ninguna reserva de memoria dinámica, la función generada resultó entre 2,2 y 11,5 veces más rápida que `regex.h` de POSIX, y sus resultados coinciden con los del módulo `re` de Python.

Palabras clave: Compiladores · Expresiones Regulares · Autómatas Finitos · Sistemas Embebidos · Minimización de Hopcroft · Código C Optimizado.

1. Introducción

En sistemas embebidos e IoT la memoria y el procesamiento son limitados, y muchas veces hay que validar cadenas de texto (identificadores, tramas de sensores, comandos por puerto serie). Hay dos formas comunes de hacerlo y las dos tienen problemas: los validadores hechos a mano con `if/switch` anidados son largos y es fácil equivocarse, y las librerías de expresiones regulares (`regex.h` de POSIX o PCRE2) gastan mucha memoria y usan asignación dinámica. Además, su tiempo de ejecución depende de la implementación: las que usan *backtracking*, como PCRE2, pueden tardar un tiempo exponencial con ciertos patrones.

Para resolver esto, proponemos **Regex2AFD**, un compilador que hace todo el trabajo pesado en tiempo de compilación (en la PC *host*) y genera una función C muy liviana, $O(n)$, que solo recorre una tabla de transiciones estática. No hace falta ningún motor de expresiones regulares en ejecución.

2. Metodología de Desarrollo

Regex2AFD se desarrolló de forma asistida por inteligencia artificial (IA), combinando dos enfoques: el Desarrollo Dirigido por Especificaciones y el Desarrollo Dirigido por Tests. El principio que ordena el trabajo es que *el equipo especifica y audita, y la IA implementa*: ningún código producido por la IA forma parte de la entrega sin haber pasado antes la suite de pruebas.

2.1. Del informe a la especificación

El punto de partida es este informe. Con lo visto en la cátedra (las fases clásicas de un compilador) el equipo definió la idea, el objetivo, el alcance, las transformaciones del motor y los escenarios de prueba que demuestran que el objetivo se cumple. De ese documento se derivaron tareas acotadas (por ejemplo, el analizador léxico, las definiciones con tabla de símbolos, el motor de autómatas), cada una con sus requisitos y su criterio de aceptación, que se entregaron a la IA como especificación.

2.2. Los tests como mecanismo de auditoría

El criterio de aceptación de cada tarea se expresó como pruebas automáticas con `pytest`. La suite completa tiene 264 pruebas y combina cuatro clases:

- **Unitarias y de integración:** cada fase y cada transformación, incluidos los mensajes de error con su posición.
- **De caracterización:** fijan el comportamiento observado del código existente antes de modificarlo, para detectar regresiones.
- **Diferenciales:** comparan el resultado contra implementaciones ajenas al código evaluado.
- **De propiedades no funcionales:** ausencia de memoria dinámica (Valgrind) y tiempo de ejecución frente a `regex.h`.

Esto da a la auditoría una base independiente de la IA: el módulo `re` de Python, `regex.h` de POSIX y Valgrind actúan como oráculos que no comparten errores con el código generado.

2.3. Alcance de la metodología

Las pruebas funcionaron como una compuerta de aceptación, dado que no seguimos un ciclo estricto de escribir primero la prueba y luego el código. Por ejemplo, en el analizador sintáctico se partió de una suite de caracterización sobre el código inicial, y las pruebas de la Sección 8 se incorporaron para auditar los módulos del motor. Además, la validación tiene un alcance acotado, ya que el fuzzing de 1500 cadenas compara el simulador del AFD (no el binario C) contra `re`, que opera sobre caracteres, mientras que el código C opera sobre bytes.

3. Arquitectura General del Compilador

El sistema sigue una serie de pasos secuenciales y conectados, clásico de un compilador, dividido en tres etapas (Fig. 1):

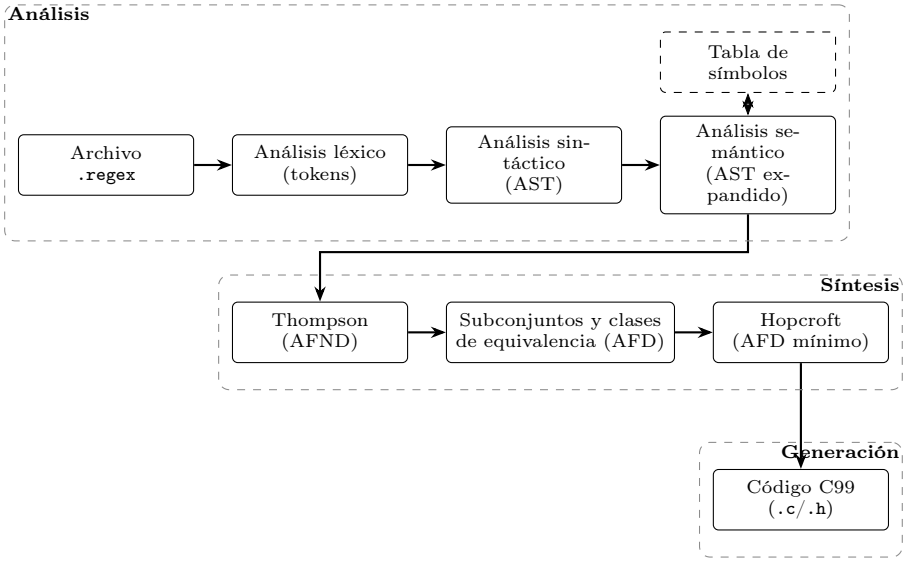


Figura 1. Pasos secuenciales de compilación de Regex2AFD en tres etapas.

1. **Frontend (Análisis):** El analizador léxico separa la expresión en tokens, el parser descendente recursivo arma el AST respetando la precedencia de los operadores, y el análisis semántico resuelve las definiciones con nombre usando una tabla de símbolos. Cada error se informa con su línea y su posición.
2. **Motor de Autómatas (Síntesis):** Pasa el AST a un AFND con Thompson y lo convierte en AFD con subconjuntos, agrupando en el mismo paso los 256 valores posibles de un byte en clases de equivalencia. Por último aplica Hopcroft, un método que reduce al mínimo el número de estados del AFD.
3. **Backend (Generación de Código):** Genera el archivo C (`.c`/`.h`) con la tabla de transiciones y el bucle de validación.

4. Especificación Sintáctica y Léxica

El compilador acepta expresiones regulares, es decir, lenguajes de Tipo 3 en la jerarquía de Chomsky. La sintaxis soportada es:

- **Literales y Concatenación:** `ab`

- **Alternancia (Unión):** `a|b`
- **Clausuras:** Kleene (`a*`), Positiva (`a+`) y Opción (`a?`)
- **Agrupamiento y Clases:** `(a|b)*`, `[a-zA-Z0-9_]`
- **Comodín:** `.` coincide con cualquier carácter salvo el salto de línea.
- **Escapes de caracteres:** `\n`, `\t` y una barra seguida de un símbolo, que lo vuelve literal (`\+`, `\.`).
- **Referencias:** `{NOMBRE}` usa una definición con nombre (ver Sección 4.2).

4.1. Gramática

El parser descendente recursivo implementa la siguiente gramática, con una función por no terminal:

$$
\begin{aligned}
\mathit{expr} &\rightarrow \mathit{subexpr}\ (\texttt{|}\ \mathit{subexpr})^* \\
\mathit{subexpr} &\rightarrow \mathit{factor}\ \mathit{factor}^* \\
\mathit{factor} &\rightarrow \mathit{átomo}\ (\texttt{*} \mid \texttt{+} \mid \texttt{?})^* \\
\mathit{átomo} &\rightarrow \texttt{CHAR} \mid \texttt{CLASS} \mid \texttt{REF} \mid \texttt{(}\ \mathit{expr}\ \texttt{)}
\end{aligned}
$$

La precedencia, de mayor a menor, es: cuantificadores, concatenación y unión. El conjunto PRIMERO de todos los no terminales es $\{\texttt{CHAR}, \texttt{CLASS}, \texttt{REF}, \texttt{(}\}$, y en cada repetición los tokens que la continúan son disjuntos de su conjunto SIGUIENTE, por lo que la gramática es LL(1). El analizador léxico agrega un token de fin de entrada que cumple el rol del símbolo $.

4.2. Análisis semántico y tabla de símbolos

Para dar nombre a las subexpresiones, la entrada admite definiciones al estilo de Flex, separadas del patrón principal por `%%`:

```
DIGITO = [0-9]
ENTERO = {DIGITO}+
NUMERO = {ENTERO}(\.{ENTERO})?
TEMPERATURA = TEMP={NUMERO}
HUMEDAD = HUM={ENTERO}
%%
{TEMPERATURA};{HUMEDAD}
```

La fase semántica registra cada definición en la tabla de símbolos y rechaza las definiciones duplicadas, las referencias sin definición, las referencias circulares (informando el camino, por ejemplo `A -> B -> A`) y los caracteres fuera del alfabeto de 256. Las definiciones que no se usan generan una advertencia. Su resultado es el AST del patrón principal con las referencias ya expandidas, que es la entrada del motor de autómatas.

5. Limitaciones y Trabajo Futuro

- **Limitaciones:** Solo sirve para patrones que se conocen al compilar. Tampoco soporta negación de clases (`[^a-z]`), cuantificadores acotados (`a{3,5}`), anclas (`^`, `$`), clases abreviadas (`\d`, `\w`) ni retroreferencias (estas últimas

no se pueden hacer con lenguajes regulares). Todas ellas producen un error léxico en lugar de interpretarse como literales, para que el comportamiento coincida con el del módulo `re` de Python.

- **Trabajo Futuro:** (i) Generar el código con `switch/case` como alternativa a las tablas; (ii) Juntar varios patrones en un solo autómata, para llegar a un generador de analizadores léxicos completo.

6. Transformaciones e Implementación del Motor

El motor aplica tres transformaciones, una después de la otra (junto con la construcción del AST y la generación de C, el pipeline completo tiene cinco):

1. **Construcción de Thompson:** Recorre el AST de forma recursiva y arma el AFND, uniendo los pedazos con transiciones épsilon (ϵ).
2. **Construcción de Subconjuntos con Clases de Equivalencia:** Antes de construir el AFD, junta los caracteres que se comportan igual en todo el autómata. Para un identificador de C (`[a-zA-Z_][a-zA-Z0-9_]*`) los 256 valores de un byte quedan en solo 3 clases (*letra*, *dígito*, *otro*). Después saca el no determinismo calculando las ϵ-clausuras, pero recorriendo esas pocas clases en lugar de 256 caracteres, y arma una tabla de transiciones más chica.
3. **Minimización de Hopcroft:** Junta los estados que no se pueden distinguir. Cada estado que se elimina achica el arreglo que se genera.

7. Generación de Código C (Backend)

El backend genera en código C estándar (C99) la tabla de transiciones y el recorrido de la cadena. Un ejemplo, para un identificador, podría verse así:

```c
/* Mapeo de los 256 valores de un byte a 3 clases de equivalencia */
static const unsigned char clase[256] = { /* generado automáticamente */ };

/* Tabla de transiciones: [estado][clase] */
static const unsigned char trans[3][3] = {
    /*          letra  digito  otro */
    /* q0 */ {    1,     2,     2 },  /* q0: estado inicial */
    /* q1 */ {    1,     1,     2 },  /* q1: estado de aceptación */
    /* q2 */ {    2,     2,     2 }   /* q2: estado de rechazo */
};

int validar_id(const char *s) {
    int estado = 0;
    while (*s != '\0') {
        estado = trans[estado][clase[(unsigned char)*s++]];
    }
    return estado == 1;
}
```

La función no usa `malloc` (asignación de memoria), no depende de nada externo y hace un solo acceso a la matriz por cada carácter.

8. Escenarios de Prueba y Evaluación

Para comprobar que el compilador funciona bien armamos pruebas que se pueden repetir. Todas forman parte de la suite de `pytest` (264 pruebas, todas aprobadas) y las dos últimas también se ejecutan con `make valgrind` y `make bench`:

1. **Pruebas Unitarias:** Cubren el análisis léxico, sintáctico y semántico, los mensajes de error con su posición, la interfaz de línea de comandos y cada transformación del motor (por ejemplo, que el identificador de C produzca exactamente las 3 clases y la tabla de 3×3 de la Sección 7). Los doce ejemplos incluidos (IPv4, URL, correo, fecha, UUID, trama de un sensor, entre otros) se verifican con cadenas que deben aceptarse y rechazarse.
2. **Validación Lógica:** Se hace *fuzzing*: se comparan los resultados del AFD mínimo con el módulo `re` de Python en 1500 cadenas aleatorias, que incluyen casos borde como la cadena vacía, saltos de línea y tabulaciones. Además, el código C generado se compila con `gcc` y se prueba directamente.
3. **Consumo de Memoria:** Se compila el código generado de cuatro ejemplos junto con un programa de prueba y se lo ejecuta con Valgrind. En todos los casos el resumen informa `0 allocs, 0 frees, 0 bytes allocated`, sin errores de memoria. El programa de prueba no usa `stdio`, porque la biblioteca de C reserva en el *heap* el buffer de la salida estándar en la primera escritura, y Valgrind contaría esa reserva aunque la función generada no haga ninguna.
4. **Desempeño Comparativo:** Se traduce el AST de cada ejemplo a una expresión regular extendida de POSIX y se mide el tiempo medio de 10^6 validaciones con la función generada y con `regexec()` de `regex.h`, sin contar la compilación de la expresión (`regcomp()`). Antes de medir se verifica que ambos clasifiquen igual 1000 cadenas por ejemplo (muestras y mutaciones aleatorias): no hubo ninguna discrepancia. La Tabla 1 muestra los resultados.

Tabla 1. Tiempo medio por cadena con 10^6 validaciones (`gcc -O2`, x86-64). Los valores absolutos dependen de la máquina.

Ejemplo	Regex2AFD	`regex.h`	Aceleración
Identificador de C	6,7 ns	76,5 ns	11,5×
IPv4	7,4 ns	62,1 ns	8,4×
Correo electrónico	15,5 ns	90,9 ns	5,9×
URL	35,8 ns	134,6 ns	3,8×
UUID	38,7 ns	83,3 ns	2,2×

La función generada fue entre 2,2 y 11,5 veces más rápida. La ventaja es mayor en los ejemplos de cadenas cortas (identificador, IPv4), lo que sugiere que allí pesa más el costo fijo de cada llamada a `regexec()`; en las cadenas largas domina el recorrido carácter por carácter de ambos.

9. Comparación con Trabajos Relacionados

Las librerías como `regex.h` o `PCRE2` trabajan en tiempo de ejecución. Regex2AFD, en cambio, sigue la idea de herramientas ya conocidas como `re2c` y `Ragel`, que hacen todo el trabajo durante la compilación (Tabla 2).

El tiempo de ejecución de `regex.h` depende de la biblioteca que lo implemente. La de glibc, por ejemplo, no usa *backtracking* salvo con retroreferencias: con patrones como `(a|aa)*b` o `(a+)+b`, que hacen explotar a los motores con *backtracking*, medimos un tiempo lineal en cadenas de hasta 16 000 caracteres. La diferencia de Regex2AFD es que el tiempo lineal está garantizado por construcción y no depende de la biblioteca de la plataforma.

Tabla 2. Comparación de Regex2AFD frente a las alternativas tradicionales.

Característica	Validador a Mano	Librería (`regex.h`)	Regex2AFD
Dependencias externas	No	Sí	**No**
Memoria dinámica	No	Sí	**No (0 bytes)**
Tiempo de ejecución	Variable	Depende de la implementación	**Lineal** $O(n)$
Mantenibilidad	Dificultosa	Alta	**Alta**

10. Conclusiones

Regex2AFD logró lo que buscábamos: una serie de pasos automatizados que se ejecutan de forma encadenada y van de la expresión regular al código C. Integramos las fases clásicas de un compilador (análisis léxico, sintáctico y semántico con tabla de símbolos, transformaciones de autómatas, minimización y generación de C). El resultado es una función en C que no usa memoria dinámica y siempre recorre la cadena en tiempo lineal, ideal para dispositivos con recursos limitados. Las pruebas lo confirman: Valgrind no registra ninguna reserva en el *heap*, y la función generada coincide con `regex.h` en todas las cadenas probadas y es entre 2,2 y 11,5 veces más rápida.

Glosario

AFD Autómata Finito Determinista. En cada estado, para cada carácter, hay un solo camino posible.

AFND Autómata Finito No Determinista. Puede tener varios caminos para un mismo carácter y transiciones épsilon.

Análisis semántico Fase que verifica el significado de la entrada, por ejemplo que cada referencia tenga una definición y que no haya definiciones circulares.

AST Árbol de Sintaxis Abstracta. Árbol que representa la estructura de la expresión regular después de analizarla.

Backtracking Técnica que prueba un camino y, si falla, vuelve atrás a probar otro. Con ciertos patrones puede tardar muchísimo.

C99 Versión del estándar del lenguaje C publicada en 1999. Se optó por usar esta porque casi todos los compiladores de microcontroladores la soportan.

Clase de equivalencia Grupo de caracteres que tienen las mismas transiciones en el autómata. Permite achicar la tabla.

Cota Límite máximo. Si el tiempo de ejecución tiene cota, se asegura que nunca pasa de cierto valor.

DSL *Domain-Specific Language*. Lenguaje o herramienta hecha para resolver un solo tipo de problema, en este caso expresiones regulares.

Embebido Sistema informático chico que está dentro de otro dispositivo y hace una tarea puntual, con poca memoria y poco procesamiento (por ejemplo, un sensor o un microondas).

ϵ-clausura Conjunto de estados a los que se llega desde un estado usando solamente transiciones épsilon (sin leer ningún carácter).

Fuzzing Prueba que consiste en generar muchísimas entradas (incluso raras) y comparar los resultados contra una referencia para encontrar errores.

Hopcroft Algoritmo que minimiza un AFD juntando los estados que no se pueden distinguir entre sí. El resultado es el AFD con menos estados posible.

Host La PC donde se ejecuta el compilador, a diferencia del dispositivo donde después corre el código generado.

IoT *Internet of Things* (Internet de las Cosas). Objetos cotidianos con sensores que se conectan a una red.

LL(1) Gramática que puede analizarse de izquierda a derecha decidiendo cada paso con un solo token de anticipación, lo que permite un parser descendente recursivo sin retroceso.

MVP *Minimum Viable Product*. Primera versión funcional con lo mínimo necesario.

$O(n)$ Tiempo lineal. El tiempo crece en proporción al largo n de la cadena: si la cadena es el doble de larga, tarda el doble.

Parser Parte del compilador que analiza el texto de la expresión regular y arma el AST.

Pipeline Serie de pasos encadenados donde la salida de uno es la entrada del siguiente.

Subconjuntos Algoritmo que convierte un AFND en un AFD. Cada estado del AFD representa un conjunto de estados del AFND.

Tabla de símbolos Estructura que guarda cada nombre definido en la entrada junto con su información (aquí, su AST, su línea y la cantidad de usos).

Thompson Algoritmo que convierte un AST en un AFND, armándolo por partes y uniéndolas con transiciones épsilon.

Valgrind Herramienta que ejecuta un programa y detecta errores de memoria. La usamos para comprobar que no hay reservas dinámicas (`malloc`).

Referencias

1. Aho, A.V., Lam, M.S., Sethi, R., Ullman, J.D.: Compilers: Principles, Techniques, and Tools. Pearson Education, 2nd edn. (2007)
2. Hopcroft, J.E., Motwani, R., Ullman, J.D.: Introduction to Automata Theory, Languages, and Computation. Addison-Wesley, 3rd edn. (2006)